

Exercícios de Funções em Python – Teste Teórico

Exercício 1.

```
def f(x):
    if x == []:
        return 0
    return x[0] + f(x[1:])
```

O que faz esta função?

Retorna uma soma de todos os elementos

$8([1, 2, 3])$

$\hookrightarrow 1 + [2, 3] \approx 1 + 2 + [3] \approx 1 + 2 + 3 = 6$

Exercício 2.

```
def g(x):
    if x == []:
        return 0
    if x[0] > 0:
        return x[0] + g(x[1:])
    return g(x[1:])
```

O que faz esta função?

Só Positivos

Retorna a soma dos números positivos da lista

Exercício 3.

```
def h(x):
    if x == []:
        return []
    return h(x[1:]) + [x[0]]
```

O que faz esta função?

Retorna a lista inversa

$\rightarrow [1, 2, 3]$

$h([2, 3]) + [1]$

$h([3]) + [2]$

$h([]) + [3]$

$[0]$

$[] + [3] = [3]$

$[3, 2] + [1] = [3, 2, 1]$

$[3] + [2] = [3, 2]$

Exercício 4.

```
def p(x):
    if x == []:
        return [[]]
    y = p(x[1:])
    return y + [[x[0]] + z for z in y]
```

O que faz esta função?

$p([1, 2, 3]) \leadsto x[0] = 1 \leadsto [1] + [], [1] + [3], [1] + [2, 3]$

1ª chamada

$y = p([2, 3]) \leadsto x[0] = 2 \leadsto [2] + [], [2] + [3]$

2ª chamada

$y = ([3]) \leadsto x[0] = 3 \leadsto [] + [3]$

3ª chamada

$y = ([]) \leadsto [] + []$

No final é juntar Tudo!

Exercício 5.

```
def q(lista, n):
    if lista == []:
        return False
    if lista[0] == n:
        return True
    return q(lista[1:], n)
```

O que faz esta função?

Verifica se na lista está um certo número (n)

True ou # False

Exercício 6.

```
def r(lista):
    if lista == []:
        return 0
    return 1 + r(lista[1:])
```

O que faz esta função?

Retorna o len (lista) - "Número Elementos"

$r([10, 20, 30])$

$\hookrightarrow r[30] = 1 + 0$

$r[20, 30] = 1 + 1$

$r[10, 20, 30] = 1 + 1 + 1 = 3$

Exercício 7.

```
def s(lista):
```

Lista com os números pares

```
if lista == []:  
    return []  
if lista[0] % 2 == 0:  
    return [lista[0]] + s(lista[1:])  
return s(lista[1:])
```

return list

O que faz esta função?

Exercício 8.

```
def t(n):  
    if n == 0:  
        return 1  
    return n * t(n - 1)
```

n!

O que faz esta função?

Exercício 9.

```
def u(lista):  
    if lista == []:  
        return []  
    return [[lista[0]]] + u(lista[1:])
```

O que faz esta função?

$[1, 2, 3] \rightarrow [[1]] + u([2, 3])$
 $[[2]] + u([3])$
 $[[3]] + u([])$
$[[1], [2], [3]]$

Exercício 10.

```
def v(x):  
    if x == []:  
        return 0  
    if isinstance(x[0], list):  
        return v(x[0]) + v(x[1:])  
    return x[0] + v(x[1:])
```

$v([1, [2, [3]], 4])$



- 1 $v([3]) = 3$
- 2 $v([[3]]) = v([3]) + v([]) = 3 + 0 = 3$
- 3 $v([2, [3]]) = 2 + v([[3]]) = 2 + 3 = 5$
- 4 $v([[2, [3]], 4]) = v([2, [3]]) + v([4]) = 5 + 4 = 9$
- 5 $v([1, [2, [3]], 4]) = 1 + 9 = 10$

O que faz esta função? Soma todos os números

Exercício 11.

```
def w(n):  
    if n <= 1:  
        return n  
    return w(n-1) + w(n-2)
```

mesmo que não estejam
todas na mesma lista

Sequência Fibonacci

O que faz esta função?

Exercício 12.

```
def z(a):  
    if a == []:  
        return 0  
    return 1 + z(a[2:])
```

count

O que faz esta função?

Conta metade do número
de elementos da lista (2 em 2)
 $a[2:]$